

Google Photos Discovery Engine — Architecture

2026-09-18 · @Someone

Production-grade architecture for an AI-powered discovery engine that surfaces retrieval-failure patterns from public user feedback, using Groq as the inference layer.

Objective & scope

The engine's job is not sentiment analysis on reviews. It exists to answer three specific questions from real user language, at scale, so Part 2 (the metric decomposition) and Part 4 (problem definition) are evidence-backed rather than assumed:

1. **What do people remember** about a photo they can't find (place, object, feeling, rough time, event) vs. **what they've forgotten** (exact date, exact words, album/folder)?
2. **Where does retrieval break** — at expressing the memory, at the system interpreting the clue, at evaluating results, or at recovering from a failed search?
3. **Which failure pattern is most frequent and most severe** (highest user effort, most abandoned searches, most workarounds), so the PM can rank opportunity areas with evidence instead of intuition.

Out of scope: general search-quality complaints (slow app, sync bugs, storage costs) — the pipeline must classify and discard these rather than count them as retrieval-failure signal.

Data sources & collection layer

Source	Method	Notes
Reddit (r/googlephotos, r/GooglePixel, r/Android)	Reddit API (PRAW) or Pushshift-style archive, keyword + subreddit filters	Richest source for specific, narrated retrieval failures ("I remember X but couldn't find it")
Play Store reviews	Play Store scraper (e.g. <code>google-play-scraper</code> npm/py lib) by app id, filtered to 1-3 star reviews mentioning search/find/remember	Reviews are short; low per-review signal, high volume — needs aggressive filtering before LLM cost is spent

Source	Method	Notes
App Store reviews	App Store RSS review feed / <code>app-store-scraper</code> , same filtering	Same treatment as Play Store
Google Photos Help Community	Site crawl of community threads (public, no auth) via a scheduled scraper respecting robots.txt	Threads often contain a full failure narrative + others' replies (workarounds) — high value
YouTube comments	YouTube Data API v3, comments on Google Photos feature/tutorial videos	Lower density of retrieval-specific complaints; useful for workaround discovery

Each collector runs as an independent, source-specific job (own rate limits, own auth, own schema quirks) writing raw records into a **landing zone** (object storage, e.g. S3/GCS) partitioned by `source/date` . Collectors never call the LLM directly — decoupling collection from analysis lets each scale and fail independently.

Ingestion & preprocessing

A lightweight pipeline runs between the landing zone and the LLM layer, cheap enough to run on every raw record before any Groq call is spent:

1. **Normalize** — unify schema across sources into one record shape: `{id, source, text, timestamp, rating?, thread_context?, url}` .
2. **Dedup** — hash-based near-duplicate removal (cross-posted Reddit threads, repeated boilerplate reviews).
3. **Language filter** — keep English (or route non-English through a translation step if multi-market coverage is wanted later).
4. **Relevance pre-filter (cheap classifier, not the main LLM)** — a small/fast model or rules-based filter (keyword + regex: "remember", "used to have", "can't find", "looking for a photo", "searched for") discards records with no retrieval-memory signal. This step matters most for Play/App Store data, where relevant reviews may be <5% of volume — without it, Groq spend is dominated by noise.
5. **Batch** — group surviving records into batches sized for the extraction layer's context window and rate limits.

Only records that pass step 4 proceed to the LLM extraction layer (s5).

Extraction schema

Every record that survives pre-filtering is passed through structured extraction (JSON-mode, schema-constrained) into these fields:

Field	Type	Purpose
<code>is_retrieval_failure</code>	bool	Confirms the record is actually about failing to find a remembered photo (final relevance gate)
<code>memory_cues_present</code>	enum[]	What the user remembered: <code>place</code> , <code>time_rough</code> , <code>time_exact</code> , <code>person</code> , <code>object</code> , <code>event</code> , <code>emotion</code> , <code>visual_detail</code> , <code>app_context</code> (album/folder)
<code>memory_cues_missing</code>	enum[]	What they explicitly say they couldn't recall: <code>exact_date</code> , <code>exact_location</code> , <code>search_keywords</code> , <code>album_name</code> , <code>who_was_in_it</code>
<code>search_behavior</code>	enum	How they tried to search: <code>keyword_guess</code> , <code>manual_scroll</code> , <code>album_browse</code> , <code>date_range_guess</code> , <code>person_filter</code> , <code>gave_up</code>
<code>failure_stage</code>	enum	Where it broke: <code>expression</code> (couldn't phrase it), <code>interpretation</code> (system misread the clue), <code>evaluation</code> (too many/wrong results to sift), <code>recovery</code> (couldn't refine after a miss)
<code>workaround_used</code>	string null	What they did instead (asked a friend, scrolled manually, used a different app, gave up)
<code>emotional_intensity</code>	1-5	Frustration/importance signal, for severity weighting
<code>outcome</code>	enum	<code>found_eventually</code> , <code>never_found</code> , <code>unclear</code>
<code>verbatim_quote</code>	string (≤25 words)	Short evidence span, for traceability back to source — never reproduced beyond this pipeline's internal use

Field	Type	Purpose
photo_type	enum	Category of the remembered photo: screenshot, document_receipt, candid_moment, posed_person_group, scenery_place, food, other — lets opportunity clusters group by photo kind, not just failure pattern

This schema is the contract between the LLM layer and everything downstream (storage, clustering, Part 2's metric tree). `failure_stage` in particular is designed to map directly onto the decomposition questions in Part 2 (expression vs. interpretation vs. evaluation vs. recovery).

LLM analysis layer (Groq)

Why Groq here: the pipeline is high-volume, low-latency-tolerant-but-cost-sensitive structured extraction over short text records — exactly the profile Groq's LPU inference is priced and optimized for (fast, cheap, high-throughput inference on open-weight models), versus a slower/costlier general-purpose API better suited to the smaller downstream synthesis step.

Two-tier model use:

- **Tier 1 — extraction (bulk, cheap):** a smaller open-weight model on Groq (e.g. Llama-3.1-8B class) run in JSON mode against the schema in s4, one call per record or small batch. This is the highest-volume, highest-cost-sensitivity step, so model size is minimized against a labeled validation set (see below).
- **Tier 2 — synthesis & comparison (low volume, higher reasoning):** a larger Groq-hosted model (e.g. Llama-3.1-70B class) run over pre-aggregated cluster summaries (not raw records) to produce the opportunity-area comparisons in s7. Low call volume, so cost is not the constraint — reasoning quality is.

Prompt design:

- System prompt fixes the schema, forbids inference beyond stated text ("do not infer emotions or facts not stated"), and includes 3–5 few-shot examples covering both memory-cue types (experiential vs. object/mundane).
- Output is constrained via Groq's JSON schema / tool-calling mode, not free text — removes parsing failures at scale.

Quality control: a held-out set of ~150–200 hand-labeled records is used to measure extraction precision/recall per field before the pipeline is trusted for Part 2 decisions; re-run against this set whenever the prompt or model changes.

Storage & indexing

- **Structured store:** extracted records (schema from s4) land in a columnar/analytical store (e.g. BigQuery or Postgres) — this is what powers the metric decomposition, filtering, and cross-source comparison in SQL.
- **Vector store:** the `verbatim_quote` plus surrounding context is embedded (Groq-compatible or separate embedding model, e.g. an open embedding model run alongside) and written to a vector index (e.g. pgvector or a managed vector DB) — this is what powers semantic clustering in s7 and later lets a PM ask free-form questions like "show me every case where a user remembered an object but not a place."
- **Raw archive:** original text + source URL retained in the landing zone (object storage) for traceability and reprocessing if the schema changes — extraction is never destructive of source data.
- **Chunking strategy:** most source posts (Reddit comments, reviews, forum replies) are short enough to embed whole — the extracted record (`verbatim_quote` + context) is embedded as one unit, not split. Long narrative threads are the exception: chunked by semantic unit (one paragraph = one attempt or workaround = one chunk) rather than a fixed token window, so each chunk stays one coherent failure moment. What's embedded is the extracted record, not raw text — this is what makes clustering group by failure pattern rather than surface wording.

Vector database choice: pgvector on Postgres is the default — keeps structured fields and embeddings in one system, and lets a query join `failure_stage` with vector similarity directly. A managed vector DB (Pinecone, Qdrant, Weaviate) is worth the extra ops only if record volume grows into the multi-million range and Postgres ANN performance becomes the bottleneck. Embeddings are produced by a compact open encoder (e.g. BGE/E5-class) run separately from the Groq extraction step, since embedding doesn't need LLM-level inference.

Clustering & opportunity comparison

This is the step that makes the engine "go beyond summarizing reviews":

1. **Cluster** records by embedding similarity within each `failure_stage` bucket (so clusters are compared apples-to-apples — e.g. clusters of *interpretation* failures, separately from clusters of *evaluation* failures), using HDBSCAN or k-means over the vector store.
2. **Label clusters** — the Tier 2 Groq model reads a sample of records per cluster and produces a short name + description (e.g. "Object-anchored memory, no place/time recall" or "Correct keyword, overwhelmed by result volume").

3. **Score clusters** on three axes pulled straight from the structured store: **frequency** (record count), **severity** (mean `emotional_intensity`, `outcome = never_found` rate), and **distinctness** (does this cluster map to a failure stage competitors/current search already handle, or a genuinely uncovered one?).
4. **Comparative output** — a ranked table of opportunity areas (cluster name, failure stage, frequency, severity, example quotes, current workaround) is the artifact Part 2 and Part 4 consume directly — not a review summary, but a ranked, evidenced set of candidate root causes.

Production architecture

```

flowchart LR
    A[Reddit API] --> E[Landing Zone<br/>object storage]
    B[Play/App Store<br/>scrapers] --> E
    C[Help Community<br/>crawler] --> E
    D[YouTube Data API] --> E
    E --> F[Preprocess:<br/>normalize, dedup]
    F --> G[Relevance<br/>pre-filter]
    G --> H[Groq Tier 1:<br/>schema extraction]
    H --> I[Structured Store<br/>BigQuery/Postgres]
    H --> J[Vector Store<br/>embeddings]
    J --> K[Clustering]
    I --> K
    K --> L[Groq Tier 2:<br/>label + synthesize]
    L --> M[Opportunity<br/>Report]
  
```

Each arrow is a decoupled, independently-scalable stage — a collector failing or a source changing its markup never blocks the others, and reprocessing after a schema change replays from the Landing Zone without re-scraping.

Orchestration, scheduling & scaling

- **Orchestrator:** Airflow (or a managed equivalent, e.g. Dagster/Prefect) schedules each collector as its own DAG — Reddit/YouTube on a daily incremental pull (only new posts since last run), Play/App Store weekly (review volume is lower-frequency signal), Help Community daily.
- **Extraction workers:** Tier 1 Groq calls run as a horizontally-scaled worker pool (queue-based, e.g. SQS/Pub-Sub-backed) consuming pre-filtered batches — Groq's inference speed means the bottleneck is usually the upstream scrape/filter stage, not extraction.

- **Rate limits & backoff:** two separate limit budgets are tracked, not one — (1) each collector respects its source's own limits (Reddit API quota, YouTube Data API quota, Play/App Store scraping etiquette — randomized delay + backoff to avoid IP blocks on daily runs) independently, so a source rate-limited or blocked degrades that source's freshness only, never halts the pipeline; (2) the Groq extraction layer is throttled separately against Groq's own per-model RPM (requests/minute) and TPM (tokens/minute) limits for whichever tier/model is used — the extraction worker pool (below) pulls from its queue at a rate capped under the model's published limit, with exponential backoff and jitter on 429s, so a burst of freshly-scraped reviews queues up rather than getting rejected. Daily scrape volume is sized against this Groq throughput ceiling: if a day's collected-and-filtered volume would exceed what the Tier 1 model can process before the next day's run, the queue carries the backlog over rather than dropping records.
- **Reprocessing:** because raw text is retained in the Landing Zone, a prompt or schema change re-runs Tier 1 extraction over historical data without re-collecting — this is what lets the schema in s4 evolve as Part 2's needs sharpen.

Observability, cost & guardrails

- **Cost control:** the relevance pre-filter (s3) is the primary cost lever — it should reject the large majority of raw Play/App Store volume before any Groq call; track "records collected" vs. "records sent to Tier 1" as a core pipeline metric.
- **Extraction monitoring:** track schema-validation failure rate, `is_retrieval_failure` positive rate per source (a sudden drop signals prompt drift or a source format change), and per-field null rates.
- **Drift checks:** re-run the held-out labeled set (s5) on a schedule (e.g. monthly) and whenever the underlying Groq model version changes — Groq's fast model iteration cycle means silent model swaps are a real risk to extraction consistency.
- **Privacy/compliance:** no usernames, emails, or other PII are retained past the raw archive's access-controlled tier; the structured store and vector store carry no user-identifying fields — only `source`, `text`, `url` (public post link) and the extracted schema.

Output layer — feeding Part 2

The engine's final deliverable is the ranked opportunity-area table from s7, queryable along the exact axes Part 2's decomposition needs:

- Filter by `failure_stage` → directly answers "is the bottleneck expression, interpretation, evaluation, or recovery?"

- Cross-tab `memory_cues_present` × `memory_cues_missing` → shows which memory-fragment types (place, object, time, emotion) correlate with which failure stage.
- `workaround_used` field → surfaces what users already do instead, for the "existing workarounds" requirement in Part 4.
- Frequency + severity scores per cluster → the evidence base for prioritizing one opportunity area over others, rather than picking one on instinct.

This table — not raw reviews, not sentiment scores — is what Part 2 and Part 4 are built on.

PM-facing interface (frontend & backend)

The pipeline stages above (s1–s11) are backend-only — they produce data, not something to look at. This layer is what you'd actually open day to day to explore insights.

Backend API (thin service layer):

- A small FastAPI (or Node/Express) service sits in front of the Structured Store and Vector Store — it never talks to Groq's bulk extraction, only to the already-processed data.
- Endpoints: `GET /clusters` (ranked opportunity list from s7, filterable by `failure_stage`, `photo_type`, source, date range), `GET /clusters/{id}` (cluster detail: description, example quotes, frequency/severity trend over time), `GET /search?q=` (semantic search — embeds the query, hits the vector store, returns matching records), `POST /ask` (free-form natural-language question).
- `/ask` is the interesting one: it takes your question ("what do people forget most when the photo is a screenshot?"), uses a Groq call to translate it into a structured filter/SQL query against the Structured Store (schema-constrained, same pattern as s5's extraction), executes it, and returns both the matching records and a short synthesized answer — this is what makes it feel conversational rather than like a BI dashboard.

Frontend (single internal user — you):

- A lightweight React/Next.js single-page app, or even a Streamlit/Retool app if speed-to-build matters more than polish (this is an internal tool for one PM, not a shipped product).
- Three views: (1) a **ranked cluster table** (sortable by frequency × severity, filter chips for `failure_stage` / `photo_type` / source), (2) a **cluster detail panel** showing example verbatim quotes, the trend over time, and related clusters, (3) a **chat/search box** wired to `/ask` and `/search` for free-form exploration when you don't know which cluster to look at yet.

- No auth complexity needed for a single-user internal tool; if this ever gets shared with other PMs, a simple SSO gate in front of the API is the only addition required.

flowchart LR

A[Structured Store] --> C[Backend API]

B[Vector Store] --> C

C --> D[Cluster Table UI]

C --> E[Cluster Detail UI]

C --> F[Chat/Search UI]

F --> |NL question| G[Groq: query translation]

G --> C

This is what turns s1-s11 from a pipeline you'd have to query manually into something you actually interact with to find insights.